MPCにおけるリワインド操作を用いたMaliciousサーバ特定手法

中川 寛大[1,2,a)]　小川 一人[2]　大東 俊博[1,2]

概要：Multi-Party Computation(MPC) はクラウド等への情報漏洩対策の 1 つとして注目されている．MPC の安全性を考える上で Malicious な攻撃者への対策は不可避な問題であり，攻撃者の存在下でも安全性を保証する多くのプロトコルが存在する．Furukawa らのプロトコルはその中で最も効率的な方式の 1 つである．本研究では Furukawa 方式の検証方式に着目した Malicious サーバ特定手法を提案する．具体的には 2 種類の検証方法によって Malicious サーバ候補をそれぞれ異なる 2 台以内に限定し，重複したサーバを Malicious サーバとする方式である．この方式は 2 段構えである．1 段目では MPC の各サーバの入出力を確認することで Malicious サーバ候補を限定する．2 段目では再開地点を特定の場所までリワインドし，非候補のサーバを通信記録サーバに用いた異なる手法により 1 段目とは異なる候補に限定する．1 段目と 2 段目の結果のアンドをとり，1 台の Malicious サーバを特定する．本方式により Malicious サーバが 1 台の場合は特定が可能である．

キーワード：マルチパーティ計算，秘密分散，改ざん，Malicious サーバ特定

Malicious Server Identification in Multi-Party Computation with Rewinding

Kandai Nakagawa[1,2,a)]　Kazuto Ogawa[2]　Toshihiro Ohigashi[1,2]

Abstract: Multi-Party Computation(MPC) is a promising approach for preventing information leakage in external computing environments, but ensuring security against malicious adversaries remains a critical challenge. The proposal by Furukawa et al. is recognized as one of the most efficient solutions. In this study, we propose a method for identifying a malicious server based on the Furukawa protocol. The method employs two distinct verification procedures, each narrowing the set of malicious server candidates to at most two servers. First, candidates are identified by verifying the inputs and outputs of MPC servers. Second, the process of computation is rewound to a checkpoint, and an alternative verification is performed using a non-candidate server as a communication-recording server, producing a different candidate set. The malicious server is identified as the server common to both candidate sets. The method successfully identifies the malicious server when one malicious server is present in the MPC system.

Keywords: Multi-party computation, Secret sharing, Tampering, Malicious server identification

1. はじめに

1.1 背景

近年のクラウドの普及に伴い，ユーザが処理をクラウドに委託するケースが増えてきている．クラウドで処理を行

[1] 東海大学, Tokai University
[2] 国立研究開発法人情報通信研究機構，NICT
[a)] 5CJNM011@tokai.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

う際の入力値として個人情報を使用する場合，平文でクラウドに入力するとユーザの情報がクラウドに漏洩してしまうことが問題となる．さらに，クラウド内部で行う処理についても秘匿すべき情報となってきている．そこでクラウドに対し，ユーザの入力情報と演算（処理）内容を秘匿する秘匿計算技術の研究開発が盛んとなっている．その中でも Multi-Party Computation(MPC) [1] は汎用性と効率性を両立しており，注目が集まっている．そこで我々は MPC に焦点を当て，高速かつ実用的な方式の実現を目指して研究している．MPC は複数のサーバ（MPC サーバ）を利用した秘匿計算技術である．

MPC の方式としては garbled-circuit ベースの MPC [2] と秘密分散ベースの MPC [3] が一般的である．前者は通信量は多いが，通信回数は定数回であり，後者は通信量は少ないが，AND 演算の回数によって通信回数が増加するという特徴がある．つまり，前者は通信帯域幅に依存しているが，通信路の遅延の影響は受けにくいので高スループットではないが低レイテンシが期待できる．一方で後者は通信帯域幅には依存していないが，通信路の遅延に影響されるため，低レイテンシではないが，高スループットが期待できる．

秘密分散ベースの MPC では，秘密分散により生成された個々の分散情報（シェア）からは元の情報の復元が不可能であるという性質（秘匿性）を利用し，各々の MPC サーバでの情報の漏洩を防いでいる．また，一部のシェアが消失しても他の（複数の）シェアから消失したシェアの復元が可能という性質 (消失復元性）を持つ秘密分散法が存在する．本研究では秘匿性と消失復元性の両方の性質を持つ MPC を使用する．これにより，各 MPC サーバを別の場所に配置することで一部の地域で災害が発生した場合などにより，一部の破損により，シェアの一部が消失した場合でも情報の保護／復元が可能となる．秘匿性と消失復元性を両立した秘密分散は，シェアの数が 3 つ以上の場合に限られるため，本研究では MPC サーバを 3 台利用した秘密分散を利用した MPC を前提とする．

MPC のサーバによる攻撃モデルとして Semi-honest モデルと Malicious モデルの 2 つのモデルが最もよく使われている．前者は攻撃者であるサーバであっても，MPC のプロトコルに逸脱した処理をすることなく，取得している情報から復元情報を得ようとする攻撃モデルである．後者はプロトコルに逸脱した処理をすることで演算結果を改ざんすることやシェアを改ざんし，その影響／結果を利用して復元情報を得ようとする攻撃モデルである．

1.2 既存研究

秘密分散ベースの MPC には Goldreich らによる GMW 方式 [4] を始まりとして多数の既存研究がある [5][6] ．

これらの中で，FLNW23 は高いスループットを保ちつつ Malicious な攻撃モデルに対して耐性がある．このプロトコルは 3 つの MPC サーバのうち 1 つが値の改ざんを行う悪意あるサーバ（Malicious サーバ）であるモデルに対応しており，Malicious サーバによるデータ改ざんを検出し，プロトコルを終了させる．

MPC ではサーバ同士が頻繁に通信する．特に，MPC の乗算や AND 演算は必然的にそのような通信を必要とする [7]．Malicious サーバは，この通信を利用して，改ざんの影響を広げる．FLNW23 は，AND 演算に検証フェーズを追加することで改ざんを検知する．但し，Malicious サーバを特定する方法については言及していない．

悪意あるサーバを特定可能な MPC プロトコルに関しては，いくつかの研究が行われている [8][9] ．これらのプロトコルはセキュリティを優先し，Malicious サーバの特定を可能にしているが，FLNW23 ほど効率的ではない．そこで先行研究として我々は FLNW23 に検証用サーバを追加することで Malicious サーバの特定を可能にした [10]. 具体的には AND 演算に着目し，既存研究の AND 演算の検証において改ざんが検知された後に AND 演算の入力値，出力値を Semi-honest な検証用サーバが取得し，再計算・比較することで Malicious サーバを特定する手法を提案した．しかし，この方式は Malicious サーバを特定しきれず，Malicious サーバ候補が 2 台になる場合が存在する．

1.3 貢献

我々は検証用サーバを用いた特定手法を拡張した新しい Malicious サーバ特定手法を提案する．検証用サーバを追加した特定手法だけでは Malicious サーバを特定できない場合が存在し，Malicious サーバ候補を 2 台以内に限定することにとどまる．そこで我々は，この特定手法を拡張し，段階的に Malicious サーバ候補を限定することで Malicious サーバを特定する機能を設計し，Malicious サーバを検知・特定できる MPC プロトコルを構築した．

改ざんが発生した場合，Malicious サーバを特定せずに改ざんのみを検知して処理を中止すると，各 MPC サーバの検証および Malicious サーバの排除が必要となるため，復旧時間が長期化する可能性がある．そのため，改ざん発生時に各 MPC サーバの処理を中止せず，特定能力を強めることで復旧時間の短縮化や改ざんの再発防止につながる．そこで本研究では改ざんの再発防止や再発時における Malicious サーバの特定を目的とし，段階的に Malicious サーバを特定するプロトコルを提案する．本研究の提案手法は 2 つの段階が存在する．1 段階目では先行研究と同様に FLNW23 にて異常を検知した際に検証用サーバを追加することで Malicious サーバ候補を 2 台以内に限定する．2 段階目では検証用サーバに加え，通信路を変更した．この段階は 1 段階目で Malicious サーバ候補を 2 台に限定し，その後リワインドし，実行する．Malicious サーバ候

© 2026 Information Processing Society of Japan

補 2 台の通信を候補外の MPC サーバを経由することで再度異常検知した際に Malicious サーバ候補を 1 段階目とは異なる 2 台以内に限定し，Malicious サーバを特定する．

2. 準備

2.1 秘密分散方式

以下では，提案方式で利用する 3 out of 3 の秘密分散方式と 2 out of 3 の複製秘密分散方式を示す．

2.1.1 3 out of 3 秘密分散方式 (Share(·))

元のデータを x，秘密分散した後の分散値（シェア）を $[\![x]\!]_1, [\![x]\!]_2, [\![x]\!]_3$ とする．また，$([\![x]\!]_1, [\![x]\!]_2, [\![x]\!]_3) \leftarrow \mathsf{Share}(x)$ とする．$[\![x]\!]_1, [\![x]\!]_2, [\![x]\!]_3$ は以下を満足するように生成される．

$$x = [\![x]\!]_1 \oplus [\![x]\!]_2 \oplus [\![x]\!]_3.$$

2.1.2 2 out of 3 複製秘密分散方式 (Share23(·))

提案方式では，3 out of 3 秘密分散法で生成された 3 つのシェア $[\![x]\!]_1, [\![x]\!]_2, [\![x]\!]_3$ を利用し 2 out of 3 秘密分散法を構成する．すなわち，$([\![x]\!]_1^{23}, [\![x]\!]_2^{23}, [\![x]\!]_3^{23}) \leftarrow \mathsf{Share}^{23}(x)$ とした時，$([\![x]\!]_1^{23}, [\![x]\!]_2^{23}, [\![x]\!]_3^{23}) = (([\![x]\!]_1, [\![x]\!]_3), ([\![x]\!]_2, [\![x]\!]_1), ([\![x]\!]_3, [\![x]\!]_2))$ とする．

提案プロトコルでは，2 out of 3 秘密分散後，$[\![x]\!]_i^{23} = ([\![x]\!]_i, [\![x]\!]_{i-1})$ が MPC サーバ i で所持される．但し，$i = 1$ の場合は $[\![x]\!]_{i-1} = [\![x]\!]_3$ である．

2.2 MPC

本研究で利用する秘密分散ベースの MPC では，平文を秘密分散し，複数の MPC サーバへ送信する．各サーバはその情報を入力値として演算し，計算結果を演算の依頼者に送信する．依頼者は各サーバから得られた演算結果を合成することで演算結果を復元することができる．

2.3 基本的な演算

MPC の基本的な演算として XOR 演算，AND 演算がある．提案方式では 2 out of 3 複製秘密分散のシェアに対する演算を行うため，2 out of 3 複製秘密分散ベースの MPC の基本的な演算を示す．また，本稿において MPC サーバ i を $P_i (i \in \{1, 2, 3\})$ と記述し，P_i は $[\![x]\!]_i^{23} = ([\![x]\!]_i, [\![x]\!]_{i-1})$ と $[\![y]\!]_i^{23} = ([\![y]\!]_i, [\![y]\!]_{i-1})$ を保持しているとする．

2.3.1 XOR（$z = x \oplus y$）演算

P_i では $[\![z]\!]_i = [\![x]\!]_i \oplus [\![y]\!]_i$，$[\![z]\!]_{i-1} = [\![x]\!]_{i-1} \oplus [\![y]\!]_{i-1}$ により $([\![z]\!]_i, [\![z]\!]_{i-1})$ を出力する．その後，全ての演算結果 $[\![z]\!]_1, [\![z]\!]_2, [\![z]\!]_3$ を XOR し，z を復元する．

2.3.2 AND（$z = x \otimes y$）演算

演算前に各 P_i は $0 = \alpha_1 \oplus \alpha_2 \oplus \alpha_3$ を満足する乱数 α_i を生成する．そして，各 P_i は $[\![z]\!]_i = ([\![x]\!]_i \otimes [\![y]\!]_i) \oplus ([\![x]\!]_i \otimes [\![y]\!]_{i-1}) \oplus ([\![x]\!]_{i-1} \otimes [\![y]\!]_i) \oplus \alpha_i$ により $[\![z]\!]_i$ を生成する．さらに，各 P_i は P_{i+1} に $[\![z]\!]_i$ を送信する．P_i では自身の持つ $[\![z]\!]_i$ と送信されてきた $[\![z]\!]_{i-1}$ のペア $([\![z]\!]_i, [\![z]\!]_{i-1})$ を 2 out of 3 秘密分散のシェア $[\![z]\!]_i^{23}$ とする．このペアは 2 out of 3 秘密分散法の性質と複製秘密分散法の性質を満たしている．

2.4 Furukawa らの方式 (FLNW23) [5]

本節では FLNW23 の検証付き演算と検証方法について示す．検証付き演算は 3 台の MPC サーバの内，1 台が Malicious であるモデルに対応しており，Malicious サーバが値を改ざんした際に検証により検知し，処理を中止する．FLNW23 の検証はクライアントから得た入力値 (3 out of 3 秘密分散のシェア) を 2 out of 3 にさらに秘密分散した入力値シェアと MPC サーバが最後に出力する出力値シェアの検証，復元ありの検証，復元無しの検証が存在する．

MPC の AND 演算は通信が必要である．その通信を介して Malicious サーバの改ざんの影響が他のサーバに広がる．そこで本稿では AND 演算に注目し，AND 演算単体の検証付き演算について示す．また，FLNW23 の検証において MPC の AND 演算に直接関係のある復元無し検証に焦点を当て，検証付き AND 演算における他の検証処理やその説明は省略する．

2.4.1 検証付き AND 演算

検証付き演算の手順について以下に示す．各入力値 x, y の 3 out of 3 秘密分散の入力値シェアを $[\![x]\!]_j, [\![y]\!]_{j'}$ とし，2 out of 3 複製秘密分散のインデックスは i とする．ただし，$j \in (1, 2, 3)$，$j' \in (1, 2, 3)$ である．また，この方式では乱数 $\alpha_{j,j',i}$ は $\alpha_{j,j',i} = F_{k_i}(id_{\alpha_{j,j'}}) \oplus F_{k_{i-1}}(id_{\alpha_{j,j'}})$ により生成される．但し，$F()$ は疑似乱数生成関数である．また，$[\![x]\!]_j, [\![y]\!]_{j'}$ のシェア $[\![[\![x]\!]_j]\!]_i, [\![[\![y]\!]_{j'}]\!]_i$ を生成するために P_i では $[\![[\![x']\!]_j]\!]_i = F_{k_i}(id_{[\![[\![x']\!]_j]\!]_i}), [\![[\![y']\!]_{j'}]\!]_i = F_{k_i}(id_{[\![[\![y']\!]_{j'}]\!]_i})$ により，$[\![[\![x']\!]_j]\!]_i, [\![[\![y']\!]_{j'}]\!]_i$ を生成する．そして，$([\![[\![x']\!]_j]\!]_1, [\![[\![x']\!]_j]\!]_2, [\![[\![x']\!]_j]\!]_3) \leftarrow \mathsf{Share}([\![x']\!]_j), ([\![[\![y']\!]_{j'}]\!]_1, [\![[\![y']\!]_{j'}]\!]_2, [\![[\![y']\!]_{j'}]\!]_3) \leftarrow \mathsf{Share}([\![y']\!]_{j'})$ である．さらに，検証付き AND 演算にて検証で使用するシェア $[\![[\![a]\!]_j]\!]_i, [\![[\![b]\!]_{j'}]\!]_i$ を $[\![[\![a]\!]_j]\!]_i = F_{k_i}(id_{[\![[\![a]\!]_j]\!]_i}), [\![[\![b]\!]_{j'}]\!]_i = F_{k_i}(id_{[\![[\![b]\!]_{j'}]\!]_i})$ により生成する．そして，$([\![[\![a]\!]_j]\!]_1, [\![[\![a]\!]_j]\!]_2, [\![[\![a]\!]_j]\!]_3) \leftarrow \mathsf{Share}([\![a]\!]_j), ([\![[\![b]\!]_{j'}]\!]_1, [\![[\![b]\!]_{j'}]\!]_2, [\![[\![b]\!]_{j'}]\!]_3) \leftarrow \mathsf{Share}([\![b]\!]_{j'})$ である．また，生成した $[\![[\![a]\!]_j]\!]_i^{23}, [\![[\![b]\!]_{j'}]\!]_i^{23}$ より，$[\![[\![c]\!]_{j,j'}]\!]_i$ を生成する．そして，$([\![[\![c]\!]_{j,j'}]\!]_1, [\![[\![c]\!]_{j,j'}]\!]_2, [\![[\![c]\!]_{j,j'}]\!]_3) \leftarrow \mathsf{Share}([\![c]\!]_{j,j'})$ である．$\mathsf{Share}([\![a]\!]_j), \mathsf{Share}([\![b]\!]_{j'}), \mathsf{Share}([\![c]\!]_{j,j'})$ の正当性を保証する検証は復元あり検証と復元無し検証を組み合わせて行われる．詳細は FLNW23 を提案した文献 [5] を参照されたい．この際，$id_{\alpha_{j,j'}}, id_{[\![[\![x']\!]_j]\!]_i}, id_{[\![[\![y']\!]_{j'}]\!]_i}, id_{[\![[\![a]\!]_j]\!]_i}, id_{[\![[\![b]\!]_{j'}]\!]_i}$ はシード値であり，k_i, k_{i-1} は疑似乱数生成関数の鍵である．全ての bit 長はセキュリティパラメータ K に依存して決定される．P_i には k_i が事前に配布されており，k_{i-1} は P_{i-1} から

© 2026 Information Processing Society of Japan

送信される．$id_{\alpha_{j,j'}}, id_{[\![[\![x']\!]_j]\!]_i}, id_{[\![[\![y']\!]_{j'}]\!]_i}, id_{[\![[\![a]\!]_j]\!]_i}, id_{[\![[\![b]\!]_{j'}]\!]_i}$ はゲートインデックスなどのユニークな値である．なお，本項の入力値 $[\![x]\!]_j, [\![y]\!]_{j'}$ を 2 out of 3 秘密分散する方法は文献 [5] を参照されたい．

Step 1 P_i で検証用シェア $[\![[\![a]\!]_j]\!]_i^{23}, [\![[\![b]\!]_{j'}]\!]_i^{23}, [\![[\![c]\!]_{j,j'}]\!]_i^{23}$，乱数シェア $[\![[\![x']\!]_j]\!]_i^{23}, [\![[\![y']\!]_{j'}]\!]_i^{23}$，乱数 $\alpha_{j,j',i}$ を生成

Step 2 クライアントは各入力値 x, y の秘密分散により入力値シェア $[\![x]\!]_j, [\![y]\!]_{j'}$ を生成し，各 MPC サーバに送信

Step 3 $[\![x]\!]_j, [\![y]\!]_{j'}$ と乱数シェア $[\![[\![x']\!]_j]\!]_i^{23}, [\![[\![y']\!]_{j'}]\!]_i^{23}$ を用いて入力値の 2 重シェア $[\![[\![x]\!]_j]\!]_i^{23}, [\![[\![y]\!]_{j'}]\!]_i^{23}$ を生成

Step 4 P_i で 2.3 節の AND 演算を実行し，演算結果のシェア $[\![[\![z]\!]_{j,j'}]\!]_i^{23}$ を生成

Step 5 AND ゲートの入力値，出力値に対し，検証用シェア $[\![[\![a]\!]_j]\!]_i^{23}, [\![[\![b]\!]_{j'}]\!]_i^{23}, [\![[\![c]\!]_{j,j'}]\!]_i^{23}$ を使用した復元無し検証を実行．検証方法は 2.4.2 項を参照

Step 6 P_i は $[\![[\![z]\!]_{j,j'}]\!]_i^{23}$ に対し，復元処理を実行し，出力値の演算結果のシェア $[\![z]\!]_i^{23}$ を生成

Step 7 クライアントは全てのシェアを収集し，$z = [\![z]\!]_1 \oplus [\![z]\!]_2 \oplus [\![z]\!]_3$ により z を復元し $z = x \otimes y$ を取得

以上の Step における $z, [\![z]\!]_{j,j'}, [\![[\![z]\!]_{j,j'}]\!]_i$ は以下により求める．また，$\alpha_{j,j',i}$ は $0 = \alpha_{j,j',i-1} \oplus \alpha_{j,j',i} \oplus \alpha_{j,j',i+1}$ を満足する．

$$
\begin{aligned}
z = {} & [\![z]\!]_{1,1} \oplus [\![z]\!]_{1,2} \oplus [\![z]\!]_{1,3} \oplus [\![z]\!]_{2,1} \oplus [\![z]\!]_{2,2} \oplus [\![z]\!]_{2,3} \\
& \oplus [\![z]\!]_{3,1} \oplus [\![z]\!]_{3,2} \oplus [\![z]\!]_{3,3}.
\end{aligned}
$$

$$
\begin{aligned}
[\![z]\!]_{j,j'} &= ([\![x]\!]_j \otimes [\![y]\!]_{j'}) \\
&= ([\![[\![x]\!]_j]\!]_1 \oplus [\![[\![x]\!]_j]\!]_2 \oplus [\![[\![x]\!]_j]\!]_3) \\
&\quad \otimes ([\![[\![y]\!]_{j'}]\!]_1 \oplus [\![[\![y]\!]_{j'}]\!]_2 \oplus [\![[\![y]\!]_{j'}]\!]_3) \\
&= (([\![[\![x]\!]_j]\!]_1 \otimes [\![[\![y]\!]_{j'}]\!]_1) \oplus ([\![[\![x]\!]_j]\!]_1 \otimes [\![[\![y]\!]_{j'}]\!]_3) \\
&\quad \oplus ([\![[\![x]\!]_j]\!]_3 \otimes [\![[\![y]\!]_{j'}]\!]_1)) \oplus (([\![[\![x]\!]_j]\!]_2 \otimes [\![[\![y]\!]_{j'}]\!]_2)) \\
&\quad \oplus ([\![[\![x]\!]_j]\!]_2 \otimes [\![[\![y]\!]_{j'}]\!]_1) \oplus ([\![[\![x]\!]_j]\!]_1 \otimes [\![[\![y]\!]_{j'}]\!]_2)) \\
&\quad \oplus \big(([\![[\![x]\!]_j]\!]_3 \otimes [\![[\![y]\!]_{j'}]\!]_3) \oplus ([\![[\![x]\!]_j]\!]_3 \otimes [\![[\![y]\!]_{j'}]\!]_2) \\
&\quad \oplus ([\![[\![x]\!]_j]\!]_2 \otimes [\![[\![y]\!]_{j'}]\!]_3)\big).
\end{aligned}
$$

$$
\begin{aligned}
[\![[\![z]\!]_{j,j'}]\!]_i = {} & ([\![[\![x]\!]_j]\!]_i \otimes [\![[\![y]\!]_{j'}]\!]_i) \oplus ([\![[\![x]\!]_j]\!]_i \otimes [\![[\![y]\!]_{j'}]\!]_{i-1}) \\
& \oplus ([\![[\![x]\!]_j]\!]_{i-1} \otimes [\![[\![y]\!]_{j'}]\!]_i) \oplus \alpha_{j,j',i}.
\end{aligned}
$$

入力値 x, y に対し 3 out of 3 の秘密分散シェアにより $x \otimes y$ を記述すると以下の通りとなる．

$$
\begin{aligned}
x \otimes y &= ([\![x]\!]_1 \oplus [\![x]\!]_2 \oplus [\![x]\!]_3) \otimes ([\![y]\!]_1 \oplus [\![y]\!]_2 \oplus [\![y]\!]_3) \\
&= \big(([\![x]\!]_1 \otimes [\![y]\!]_1) \oplus ([\![x]\!]_1 \otimes [\![y]\!]_2) \oplus ([\![x]\!]_1 \otimes [\![y]\!]_3)\big) \\
&\quad \oplus \big(([\![x]\!]_2 \otimes [\![y]\!]_1) \oplus ([\![x]\!]_2 \otimes [\![y]\!]_2) \oplus ([\![x]\!]_2 \otimes [\![y]\!]_3)\big) \\
&\quad \oplus \big(([\![x]\!]_3 \otimes [\![y]\!]_1) \oplus ([\![x]\!]_3 \otimes [\![y]\!]_2) \oplus ([\![x]\!]_3 \otimes [\![y]\!]_3)\big).
\end{aligned}
$$

2.4.2 復元無し検証

この復元無し検証は，入力値 x, y（厳密には x, y のシェア $[\![x]\!]_1, [\![x]\!]_2, [\![x]\!]_3, [\![y]\!]_1, [\![y]\!]_2, [\![y]\!]_3$）と，出力値 z（厳密には z と $[\![z]\!]_1, [\![z]\!]_2, [\![z]\!]_3$）を検証対象とし，それらの関係の正当性を検証する．復元無し検証では検証用データであり $c = a \otimes b$ の関係がある a, b, c のシェアの一部である $[\![a]\!]_i^{23} [\![b]\!]_i^{23}, [\![c]\!]_i^{23}$ と，入力値 x, y のシェアの一部 $[\![x]\!]_i^{23}, [\![y]\!]_i^{23}$，出力値 z のシェアの一部 $[\![z]\!]_i^{23}$ を使用し，P_i で $[\![v]\!]_i^{23}$ を生成する．全入出力データが正しければ，$0 = [\![v]\!]_{i-1} \oplus [\![v]\!]_i \oplus [\![v]\!]_{i+1}$ となる．$[\![v]\!]_i^{23}$ の生成方法を以下に示す．本項で示す式において $h \in \{i, i-1\}$ とする．

Step 1 P_i は $[\![p]\!]_i^{23}, [\![q]\!]_i^{23}$ を以下により生成

$$[\![p]\!]_h = [\![a]\!]_h \oplus [\![x]\!]_h. \tag{1}$$

$$[\![q]\!]_h = [\![b]\!]_h \oplus [\![y]\!]_h. \tag{2}$$

Step 2 P_i が $[\![p]\!]_{i+1}, [\![q]\!]_{i+1}$ を P_{i-1} から受信

Step 3 各 MPC サーバで以下により p, q を復元

$$p = [\![p]\!]_1 \oplus [\![p]\!]_2 \oplus [\![p]\!]_3. \tag{3}$$

$$q = [\![q]\!]_1 \oplus [\![q]\!]_2 \oplus [\![q]\!]_3. \tag{4}$$

Step 4 P_i は P_{i+1} に p, q を送信

Step 5 P_{i-1} は送信された p, q と自身が生成した p, q を比較

Step 6 P_i は以下 $[\![v]\!]_i^{23}$ を計算

$$[\![v]\!]_h = [\![z]\!]_h \oplus [\![c]\!]_h \oplus (q \otimes [\![a]\!]_h) \oplus (p \otimes [\![b]\!]_h) \oplus (p \otimes q). \tag{5}$$

Step 7 P_i は P_{i+1} に $[\![v]\!]_{i-1}$ を送信

Step 8 P_i は $[\![v]\!]_i \oplus [\![v]\!]_{i-1} = [\![v]\!]_{i+1}$ を検証

2.4.1 項に示したように，AND 演算を検証する際，9 つの AND 演算があり，検証対象のシェアが変わるごとに異なる検証用シェア $a_i, a_{i-1}, b_i, b_{i-1}, c_i, c_{i-1}$ を使用する．

2.4.3 Dalskov らの方式 (DEK2021) [11]

改ざんに対して中止以外のアプローチをとりつつ Malicious サーバに耐性のあるプロトコルも存在する．一例として 30th USENIX Security Symposium にて発表されている Dalskov らの方式 (DEK2021) [11] がある．このプロトコルは Malicious サーバが改ざんした後も処理を継続するプロトコルであり，4 台の内，Malicious サーバが 1 台存在する場合を想定している．具体的にはサーバを 4 台使用した MPC プロトコルにより，改ざん発生時に Malicious サーバ候補を 2 台に限定する．その後，Malicious サーバ候補を 1 台含んだ 3 台でプロトコルを動作することにより，再発時に Malicious サーバを特定する機能を有している．

3. 提案

本稿では 1 台の Malicious サーバに耐性のある MPC プロトコルを提案する．このプロトコルは 2.4.3 項の DEK2021 [11] の様に Malicious サーバが改ざんした後

© 2026 Information Processing Society of Japan

も継続するプロトコルである．本手法は 2 段階に分かれている．1 段階目は改ざんを検知した際に Malicious サーバ候補を 2 台以内に限定する段階である．2 段階目は改ざん検知エラーが起こらなかった最後の地点までリワインドし，処理を再開する．そこで再度改ざんが発生した際に 1 段階目と異なる通信方法を用いて，Malicious サーバ候補を 1 段階目とは異なる 2 台以内に限定する段階である．この 2 段階のプロトコルによって異なる 2 つの Malicious サーバ候補の集合を作り，重複しているサーバを Malicious サーバと断定する．また，提案手法は FLNW23 に Semi-honest な検証用サーバを追加し，それによる検証機能を追加，さらに，通信経路の異なる 2 種類の MPC 方法を利用して実現している．提案手法を以下に示す．

3.1 Malicious サーバの特定：第 1 段階

第 1 段階では，異常検知時に Semi-honest な検証用サーバを追加することで Malicious サーバの候補を 2 台以内に限定する．具体的には検証用サーバにて改ざん発生時の各 MPC サーバの演算を再計算し，各 MPC サーバから収集した出力値と再計算で導出した出力値を比較することにより，改ざんした可能性のある MPC サーバを 2 台以内に限定する．本節では通信が発生する AND 演算に着目しているため，MPC の検証付き AND 演算にて異常が発生した際の検証用サーバの処理を示す．

3.1.1 検証付き AND 演算における限定手法 [10]

この手法は 2026 年 1 月に SCIS2026 にて提案した手法であり，2.4.2 項の復元無し検証において異常を検知した際に実行する．この手法は各 MPC サーバから検証用サーバに各 MPC サーバで行われた AND 演算に必要な情報を送信し，検証用サーバ内で各 MPC サーバの AND 演算を再計算する．その後，再計算で得た出力値と各 MPC サーバから得た出力値を比較することで Malicious サーバ候補を 2 台以内に限定する手法である．この際に 1 台に特定できる場合と候補を 2 台に限定できる場合の 2 つの場合が存在するが，詳細は文献 [10] を参照されたい．

第 1 段階のプロトコルを以下に示す．本プロトコルは検証用サーバで実施される．本プロトコルにおいて Malicious サーバの候補が 2 台に限定された場合，3.2 節に示す第 2 段階に進む．

プロトコル 1.

Step 1 全ての MPC サーバが所持している全ての 2 out of 3 複製秘密分散のシェア $[\![x]\!]_j]\!]_1^{23}, [\![x]\!]_j]\!]_2^{23}, [\![x]\!]_j]\!]_3^{23}, [\![y]\!]'_j]\!]_1^{23}, [\![y]\!]'_j]\!]_2^{23}, [\![y]\!]'_j]\!]_3^{23}, [\![z]\!]_{j,j'}]\!]_1^{23}, [\![z]\!]_{j,j'}]\!]_2^{23}, [\![z]\!]_{j,j'}]\!]_3^{23}$，疑似乱数 $\alpha_{j,j',1}, \alpha_{j,j',2}, \alpha_{j,j',3}$ を生成する全ての $(F_{k_1}(id), F_{k_3}(id)), (F_{k_2}(id), F_{k_1}(id)), (F_{k_3}(id), F_{k_2}(id))$ を取得

Step 2 全ての $[\![x]\!]_j, [\![y]\!]_{j'}$ の 3 out of 3 秘密分散のシェア，$(F_{k_i}(id), F_{k_{i-1}}(id))$ について後述する 3.1.2 項のプロトコル 1.1. を実行

Step 3 2.3 節の AND 演算によって演算結果のシェア $[\![z]\!]_{j,j'}]\!]_1^{23}, [\![z]\!]_{j,j'}]\!]_2^{23}, [\![z]\!]_{j,j'}]\!]_3^{23}$ を生成

Step 4 全ての $[\![z]\!]_{j,j'}]\!]_i^{23}$ のペアについて比較し，Malicious サーバ候補が 2 台になる場合は第 2 段階へ移行し，1 台に特定された場合は終了．詳細は [10] を参照

3.1.2 第 1 段階における入力データの検証

プロトコル 1. にて再計算で使用する値の正当性を保証する入力データの検証を示す．このプロトコルはプロトコル 1. の Step 2 で呼び出されるサブプロトコルであり，この検証で異常が検知されなかった場合，プロトコル 1. の Step 3 に進む．

このプロトコルでは 2 out of 3 秘密分散のシェアなど 3 台の MPC サーバの内，2 台もしくは全ての MPC サーバが共通で所持しているデータの整合性を検証する．これは AND 演算に着目した入力値の検証であり，誤った値が検証用サーバに送信された場合，Malicious サーバ候補を 2 台に限定する．プロトコルの手順を以下に示す．このプロトコルにおける検証用サーバが P_{i-1}, P_i, P_{i+1} の内，検証対象の MPC サーバ P_i, P_{i-1} から受信する検証対象の情報を in とする．

プロトコル 1.1.

Step 1 検証用サーバが P_{i-1}, P_i のいずれか 2 台から in を受信し，二つの in を比較

Step 2 異なる場合，それらの値を送信した 2 台を Malicious サーバ候補に限定

3.2 Malicious サーバの特定：第 2 段階

第 2 段階では，第 1 段階で Malicious サーバの候補となった 2 台から 1 台に絞り込む．正確には，第 1 段階での限定手法とは異なる手法により，第 1 段階で限定された 2 台とは異なる 2 台となるようにしている．そして，第 1 段階と第 2 段階でそれぞれ限定された 2 台のアンドをとることで Malicious サーバを 1 台に特定する手法である．

第 1 段階において Malicious サーバを特定できない原因は Malicious サーバ候補間の通信を確認できないことである．それにより，Malicious サーバ候補間の通信における改ざんのタイミングが送信する前か受信した後かを区別できず，否認防止性が満たされていない．そこで第 2 段階では，否認防止性を満たすべく，第 1 段階で Malicious サーバ候補から外れた MPC サーバを中継サーバとして候補間の通信ログを記録する手法を提案した．この通信では検証用サーバと Malicious サーバ候補 2 台が共有した共通鍵で暗号化し，通信する．以後，候補のサーバを P_{i-1}, P_i とし，中継サーバを P_{i+1}，Malicious サーバを P_m と記述する．さらに，P_{i-1}, P_i 間でデータを暗号化し，通信を行う際に使用する共通鍵を sk_{i-1} とする．

© 2026 Information Processing Society of Japan

検証用サーバでは第 1 段階と同様に再計算も行うが，それだけでなく，通信ログを用いた検証も行う．最初の MPC サーバ間の通信は疑似乱数生成関数の鍵の共有である．2 段階目の P_{i-1} が所持している疑似乱数生成関数の鍵 k_{i-1} を P_i に送信するときの通信の処理手順を以下に示す．

通信処理 1.

手順 1 P_{i-1} は k_{i-1} を共通鍵 sk_{i-1} を用いて $\mathsf{Enc}_{sk_{i-1}}(k_{i-1})$ に暗号化し，P_{i+1} に送信

手順 2 P_{i+1} は $\mathsf{Enc}_{sk_{i-1}}(k_{i-1})$ の通信ログを記録しつつ P_i に送信

手順 3 P_i は sk_{i-1} を用いて復号し，k_{i-1} を生成

以下では，検証付き演算において AND 演算のプロトコルと復元無し検証プロトコルと復元無し検証で異常を検知した場合の検証用サーバのプロトコルについて示す．

3.2.1 検証付き AND 演算プロトコル

第 2 段階における検証付き AND 演算を示す．このプロトコルは否認防止性を満たすために 1 段階目の限定手法で Malicious サーバ候補外とした MPC サーバで候補間の通信ログを記録する．それにより，異常検知時の検証に Malicious サーバを特定できない事態を防いでいる．

以下では，2 台の Malicious サーバ候補間の通信である P_{i-1} から P_i の通信部分を示すが，各 MPC サーバが $[\![x]\!]_j, [\![y]\!]_{j'}$ を受信し，P_i で $[\![[\![a]\!]_j]\!]_i^{23}, [\![[\![b]\!]_{j'}]\!]_i^{23}, [\![[\![c]\!]_{j,j'}]\!]_i^{23}, [\![[\![x']\!]_j]\!]_i^{23}, [\![[\![y']\!]_{j'}]\!]_i^{23}, \alpha_{j,j',i}$ を生成した後の処理となる．

プロトコル 2.

Step 1 P_i で $[\![[\![x]\!]_j]\!]_i^{23}, [\![[\![y]\!]_{j'}]\!]_i^{23}$ を生成

Step 2 P_{i-1} は所有している 2 重シェア $[\![[\![x]\!]_j]\!]_{i-1}^{23}, [\![[\![y]\!]_{j'}]\!]_{i-1}^{23}$，乱数 $\alpha_{j,j',i-1}$ を利用し，以下の式により，$[\![[\![z]\!]_{j,j'}]\!]_{i-1}$ を生成

$$\begin{aligned}[\![[\![z]\!]_{j,j'}]\!]_{i-1} = &([\![[\![x]\!]_j]\!]_{i-1} \otimes [\![[\![y]\!]_{j'}]\!]_{i-1}) \\ &\oplus ([\![[\![x]\!]_j]\!]_{i+1} \otimes [\![[\![y]\!]_{j'}]\!]_{i-1}) \\ &\oplus ([\![[\![x]\!]_j]\!]_{i-1} \otimes [\![[\![y]\!]_{j'}]\!]_{i+1}) \oplus \alpha_{j,j',i-1}.\end{aligned}$$

Step 3 P_{i-1} は $[\![[\![z]\!]_{j,j'}]\!]_{i-1}$ を共通鍵 $sk_{j,j',i-1}$ にて暗号化し，$\mathsf{Enc}_{sk_{j,j',i-1}}([\![[\![z]\!]_{j,j'}]\!]_{i-1})$ を生成し，P_{i+1} を経由しつつ P_i に送信

Step 4 P_i は自身の $\mathsf{Enc}_{sk_{j,j',i-1}}([\![[\![z]\!]_{j,j'}]\!]_{i-1})$ を共通鍵 $sk_{j,j',i-1}$ で復号し，$[\![[\![z]\!]_{j,j'}]\!]_{i-1}$ を生成

Step 5 P_i は自身の $[\![[\![z]\!]_{j,j'}]\!]_i$ と $[\![[\![z]\!]_{j,j'}]\!]_{i-1}$ を組み合わせて出力シェア $[\![[\![z]\!]_{j,j'}]\!]_i^{23}$ を生成

Step 6 P_i は検証対象のシェアを $[\![[\![x]\!]_j]\!]_i^{23}, [\![[\![y]\!]_{j'}]\!]_i^{23}, [\![[\![z]\!]_{j,j'}]\!]_i^{23}$，検証用シェアを $[\![[\![a]\!]_j]\!]_i^{23}, [\![[\![b]\!]_{j'}]\!]_i^{23}, [\![[\![c]\!]_{j,j'}]\!]_i^{23}$ とした 3.2.2 項のプロトコル 2.1.（復元無し検証）を実行

Step 7 P_i は $[\![[\![z]\!]_{j,j'}]\!]_i^{23}$ に対し，復元処理を実行し，出力値の演算結果のシェア $[\![z]\!]_i^{23}$ を生成

Step 8 クライアントは全てのシェアを収集し，$z = [\![z]\!]_1 \oplus [\![z]\!]_2 \oplus [\![z]\!]_3$ により z を復元し，$z = x \otimes y$ を取得

なお，P_{i+1} と P_{i-1} 間，P_i と P_{i+1} 間の通信については平文で通信される．そのため，これらの通信については第 1 段階と同じ検証方法が可能となる．Malicious サーバがそれらの間の通信で改ざんした場合は異なる Malicious サーバ候補に限定できる．それにより，Malicious サーバを 1 台に特定できる．

3.2.2 復元無しの検証プロトコル

2 段階目の復元無しの検証やその検証で異常を検知した際に検証用サーバを使用し，Malicious サーバを特定する手法を記述する．通信路を変更した復元無し検証をプロトコル 2.1.，検証用サーバにて P_m を特定するプロトコルをプロトコル 2.1.1.，プロトコル 2.1.2. とする．プロトコル 2.1. はプロトコル 2. のサブプロトコル，プロトコル 2.1.1.,2.1.2. はプロトコル 2.1. のサブプロトコルである．これらのプロトコルを以下に示す．

プロトコル 2.1.

このプロトコルは否認防止性を満たすために候補外のサーバを利用して候補間の通信ログを記録・検証するプロトコルである．第 2 段階における復元無し検証プロトコルを示す．以下では，P_i, P_{i-1} 間のプロトコルを示す．P_{i+1} と P_{i-1} 間，P_i と P_{i+1} 間の通信は 2.4.2 項の復元無し検証と同じである

Step 1 P_{i-1} において $[\![p]\!]_{i-1}^{23}, [\![q]\!]_{i-1}^{23}$ を以下の式と式 (1)(2) により生成し，$[\![p]\!]_{i+1}, [\![q]\!]_{i+1}$ を P_{i+1} に送信

$$[\![p]\!]_{i+1} = [\![a]\!]_{i+1} \oplus [\![x]\!]_{i+1}.$$
$$[\![q]\!]_{i+1} = [\![b]\!]_{i+1} \oplus [\![y]\!]_{i+1}.$$

Step 2 P_{i+1} は自身の所持している $[\![p]\!]_{i+1}, [\![q]\!]_{i+1}$ と受信した $[\![p]\!]_{i+1}, [\![q]\!]_{i+1}$ を比較

Step 3 異なる場合は $P_m = P_{i-1}$ として中止サイン $\perp$ を出力，等しい場合は $[\![p]\!]_{i+1}, [\![q]\!]_{i+1}$ を P_i に送信

Step 4 各 MPC サーバで式 (3)(4) により p, q を復元

Step 5 P_{i-1} が p, q を P_{i+1} に送信

Step 6 P_{i+1} は自身の所持している p, q と受信した p, q を比較．異なる場合は 2 つの p, q で以下の式により，$[\![p]\!]_{i-1}, [\![q]\!]_{i-1}$ を再計算し，検証用サーバに送信．その後，中止サイン $\perp$ を出力し，後述するプロトコル 2.1.1. を実行．等しい場合は p, q を P_i に送信

$$[\![p]\!]_{i-1} = p \oplus [\![p]\!]_i \oplus [\![p]\!]_{i+1}.$$
$$[\![q]\!]_{i-1} = q \oplus [\![q]\!]_i \oplus [\![q]\!]_{i+1}.$$

Step 7 P_i は受信した p, q と自身が生成した p, q を比較し，異なる場合は各 MPC サーバに中止サイン $\perp$ を送信．それを受け取った P_{i+1} は $P_m = P_i$ を出力

© 2026 Information Processing Society of Japan

Step 8　P_{i-1} は式 (5) により $[\![v]\!]^{23}_{i-1}$ を計算し，$[\![v]\!]_{i+1}$ を P_{i+1} に送信

Step 9　P_{i+1} は自身の所持している $[\![v]\!]_{i+1}$ と受信した $[\![v]\!]_{i+1}$ を比較し，異なる場合は $P_m = P_{i-1}$ として中止サイン $\perp$ を出力し，等しい場合は $[\![v]\!]_{i+1}$ を P_i に送信

Step 10　P_i は $[\![v]\!]_i \oplus [\![v]\!]_{i-1} = [\![v]\!]_{i+1}$ を検証し，異なる場合は後述するプロトコル 2.1.2. により，P_m を特定

プロトコル 2.1.1.

本プロトコルは，プロトコル 2.1. の Step 6 にて異常を検知した場合に検証用サーバで行われるプロトコルである．

Step 1　P_{i-1}, P_i が所持している $(p, q), ([\![x]\!]_{i-1}, [\![y]\!]_{i-1})$，$P_{i+1}$ が所持している $id_a, id_b, \mathsf{Enc}_{sk_{i-1}}(k_{i-1})$，$P_{i-1}$ から送信された $[\![p]\!]_{i-1}, [\![q]\!]_{i-1}$，プロトコル 2.1. の Step 6 で再計算した $[\![p]\!]_{i-1}, [\![q]\!]_{i-1}$ を取得

Step 2　取得した値の内，$[\![p]\!]_{i-1}, [\![q]\!]_{i-1}$ 以外で重複しているものに対して 3.2.3 項のプロトコル 2.1.3. を実行

Step 3　$\mathsf{Enc}_{sk_{i-1}}(k_{i-1})$ を共通鍵 sk_{i-1} を使用して復号し，k_{i-1} を生成

Step 4　以下の式により，$[\![a]\!]_{i-1}, [\![b]\!]_{i-1}$ を生成

$$[\![a]\!]_{i-1} = (F_{k_{i-1}}(id_a)).$$
$$[\![b]\!]_{i-1} = (F_{k_{i-1}}(id_b)).$$

Step 5　P_{i+1} から収集した 2 つの $[\![p]\!]_{i-1}, [\![q]\!]_{i-1}$ と入力データの検証を通過した $[\![x]\!]_{i-1}, [\![y]\!]_{i-1}$ を使用し，以下の式により，$[\![a]\!]_{i-1}, [\![b]\!]_{i-1}$ を再計算

$$[\![a]\!]_{i-1} = [\![p]\!]_{i-1} \oplus [\![x]\!]_{i-1}.$$
$$[\![b]\!]_{i-1} = [\![q]\!]_{i-1} \oplus [\![y]\!]_{i-1}.$$

Step 6　Step 4 で生成した $[\![a]\!]_{i-1}, [\![b]\!]_{i-1}$，Step 5 で生成した $[\![a]\!]_{i-1}, [\![b]\!]_{i-1}$ を比較し，Step 4 で生成した $[\![a]\!]_{i-1}, [\![b]\!]_{i-1}$ と異なる値を P_{i+1} に送信した MPC サーバを P_m と特定

プロトコル 2.1.2.

本プロトコルは，プロトコル 2.1. の Step 10 にて異常を検知した際に検証用サーバで行われるプロトコルである．

Step 1　全ての MPC サーバが所持している全ての 2 out of 3 複製秘密分散のシェア，疑似乱数 α_{i-1} を生成する全ての乱数 $(F_{k_{i-1}}(id), F_{k_{i+1}}(id))$，$P_{i+1}$ から P_{i-1} が生成した $[\![z]\!]_{i-1}$ の暗号文 $\mathsf{Enc}_{sk_{i-1}}([\![z]\!]_{i-1})$ を取得

Step 2　$[\![x]\!]^{23}_{i-1}, [\![y]\!]^{23}_{i-1}$ や $(F_{k_{i-1}}(id), F_{k_{i+1}}(id))$ に対して 3.2.3 項のプロトコル 2.1.3. を実行

Step 3　再計算によって演算結果のシェア $[\![z]\!]_{i-1}$ を生成

Step 4　$\mathsf{Enc}_{sk_{i-1}}([\![z]\!]_{i-1})$ を共通鍵 sk_{i-1} を用いて $[\![z]\!]_{i-1}$ に復号

Step 5　Step 4 で生成した $[\![z]\!]_{i-1}$ 以外の全ての $[\![z]\!]_{i-1}$ に対して 3.1.1 項のプロトコル 1. の Step 4 と同じ比較を実行

Step 6　もし Step 5 の比較にて異常を検知した場合，P_m を特定できれば中止．できなければ Step 7 を実行

Step 7　Step 4 において復号した通信ログ $[\![z]\!]_{i-1}$ と再計算で生成した $[\![z]\!]_{i-1}$ を比較し，異なれば P_m を P_{i-1} と特定．等しければ P_m を P_i と特定

3.2.3　第 2 段階における入力データの検証

第 2 段階における入力データの検証を示す．このプロトコルはプロトコル 2.1.1. とプロトコル 2.1.2. のサブプロトコルであり，3.1.2 項の入力データの検証に特定機能を追加したものである．2.1.1.，2.1.2. それぞれのプロトコルの Step2 から呼び出され，この検証で異常が検知されなかった場合はそれぞれのプロトコルの Step 3 に進む．

第 2 段階の入力データの検証では P_i, P_{i-1} から送信された検証対象の値を in_{i-1} とし，in_{i-1} が異なる場合，P_{i-1} 内での in_{i-1} の生成に関係する通信処理を全てさかのぼり，再計算を行う．この場合の演算内容を $f()$，演算の入力値群の内，P_{i-1} から取得したデータ群を $bf\text{-}in_{i-1}$，P_i から取得したデータ群を $bf\text{-}in_i$ と記述し，$f(bf\text{-}in_i) = in_{i-1}, f(bf\text{-}in_{i-1}) = in_{i-1}$ とする．また，このプロトコルでは P_i が $bf\text{-}in_i$ を共有するために P_{i-1} から収集した通信ログ群を $\mathsf{Enc}_{sk_{i-1}}(bf\text{-}in'_i)$ とする．

プロトコル 2.1.3.

Step 1　検証用サーバが P_{i-1}, P_i から $bf\text{-}in_{i-1}, bf\text{-}in_i, in_{i-1}$，$P_{i+1}$ から $\mathsf{Enc}_{sk_{i-1}}(bf\text{-}in'_i)$ を受信し，$\mathsf{Enc}_{sk_{i-1}}(bf\text{-}in'_i)$ を sk_{i-1} を用いて $bf\text{-}in'_i$ に復号

Step 2　P_i, P_{i-1} から受信した in_{i-1} を比較し，改ざんの有無を検証

Step 3　Step 2 の比較が異なる場合，$bf\text{-}in_{i-1}, bf\text{-}in_i$ を比較し，その比較が等しければ in_{i-1} を再計算し，再計算結果と異なる値を送信したサーバを P_m と特定

Step 4　Step 3 の比較が異なる場合，$bf\text{-}in_{i-1}, bf\text{-}in_i$，$bf\text{-}in'_i$ を比較する．$bf\text{-}in_{i-1}$ のみが異なる場合は P_{i-1} を P_m と特定し，$bf\text{-}in_i$ のみが異なる場合は P_i を P_m と特定

4.　通信路変更の影響に関する考察

本提案方式は Malicious サーバを特定する方式であり，既存の改ざん検知方式である FLNW23 に本機能を追加して利用する．そこで，本章ではこの本提案機能について通信回数や通信量への影響を考察する．

第 1 段階の MPC サーバ間の通信路を図 1 に，第 2 段階の通信路を図 2 に示す．図 1，図 2 から全体の通信回数として非候補サーバが中継することによって第 2 段階では第 1 段階と比較して三者間の通信が 1 回増えていることが確認できる．それによる影響について第 1 段階と第 2 段階の両方で Malicious サーバ候補とされている 2 台の MPC サーバは 1 台当たりの送受信の回数は変わらず，第 1 段階，

© 2026 Information Processing Society of Japan

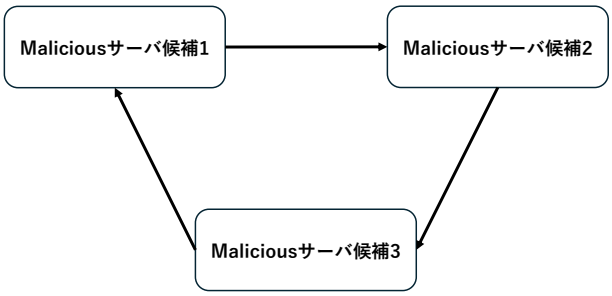


図 1　第 1 段階の通信路

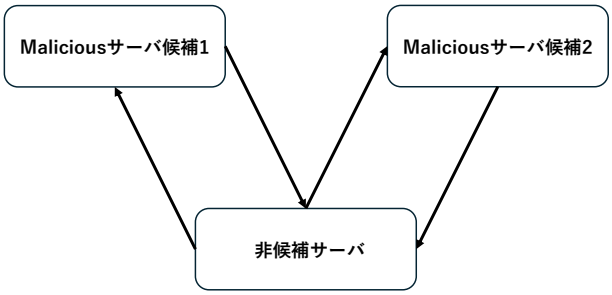


図 2　第 2 段階の通信路

第 2 段階における通信量や通信回数の変化はないと考察できる．候補外の 1 台の MPC サーバは 1 台当たりの送受信回数が FLNW23 と比較し，2 倍に収まっており，通信量や通信回数も約 2 倍に収まることが考察できる．

秘密分散ベースの MPC において通信はボトルネックとなっており，全体の処理時間に大きな影響を与える．さらに，2 台のサーバ間での通信において遅延が発生した際には残りの 1 台もその影響を受け，処理が遅延することが考えられる．本研究の第 2 段階での通信路の変更も同様に非候補サーバへの通信回数や通信量の影響が全体の処理時間に対して影響すると考えられるが，通信回数や通信量は約 2 倍収まっているため，全体の処理時間への影響も約 2 倍に収まると考えられる．

5. まとめ

先行研究として我々は FLNW23 の検証の後に検証用サーバを 1 台追加することで Malicious サーバの特定を可能にした [10]．しかし，この方式は Malicious サーバを特定しきれず，Malicious サーバ候補が 2 台になる場合が存在する．そこで本研究では特定できない場合を無くすべく，先行研究の手法を拡張し，段階的に Malicious サーバを特定可能とする機構を組み込むことで，その実用性をさらに向上させた．第 1 段階は FLNW23 において異常検知した際に検証用サーバで再計算することで，Malicious サーバ候補を限定する手法とした．第 2 段階は検証用サーバに加えて Malicious サーバ候補から外れたサーバが Malicious サーバ候補のサーバ間の通信を中継することで通信ログを記録し，第 1 段階の候補を踏まえて Malicious サーバを特定する手法とした．さらに，第 1 段階から第 2 段階への通信路の変更による処理時間への影響は約 2 倍に収まると考察した．以上の結果より，MPC の実用性をさらに高めることができたと考えている．今後は本研究の提案手法に対する安全性の証明を行う．

参考文献

[1] Y. Lindell, “Secure multiparty computation,” Commun. ACM, vol.64, no.1, p.86–96, Dec. 2020.
[2] A.C.-C. Yao, “How to generate and exchange secrets,” 27th Annual Symposium on Foundations of Computer Science (sfcs 1986), pp.162–167, 1986.
[3] M. Ben-Or, S. Goldwasser, and A. Wigderson, “Completeness theorems for non-cryptographic fault-tolerant distributed computation,” Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing, p.1–10, STOC ’88, Association for Computing Machinery, New York, NY, USA, 1988.
[4] O. Goldreich, S. Micali, and A. Wigderson, “How to play any mental game or a completeness theorem for protocols with honest majority,” STOC1987, ACM, pp.218–229, 1987.
[5] J. Furukawa, Y. Lindell, A. Nof, and O. Weinstein, “High-throughput secure three-party computation with an honest majority,” Journal of Cryptology, vol.36, no.3, p.68, 2023.
[6] Q. Dao, Y. Ishai, A. Jain, and H. Lin, “Multiparty homomorphic secret sharing and sublinear MPC from sparse LPN,” Cryptology ePrint Archive, Paper 2023/1593, 2023.
[7] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, “Multiparty computation from somewhat homomorphic encryption,” Advances in Cryptology – CRYPTO 2012, eds. by R. Safavi-Naini and R. Canetti, pp.643–662, Springer Berlin Heidelberg, Berlin, Heidelberg, 2012.
[8] R. Cunningham, B. Fuller, and S. Yakoubov, “Catching mpc cheaters: Identification and openability,” Information Theoretic Security, ed. by J. Shikata, pp.110–134, Springer International Publishing, Cham, 2017.
[9] N. Brandt, S. Maier, T. Müller, and J. Müller-Quade, “On the correlation complexity of mpc with cheater identification,” Financial Cryptography and Data Security, eds. by F. Baldimtsi and C. Cachin, pp.129–146, Springer Nature Switzerland, Cham, 2024.
[10] 中川寛大，小川一人，大東俊博，“Malicious サーバ特定手法をもつ 3-Party MPC の効率化，” 2026 年暗号と情報セキュリティシンポジウム (SCIS 2026)，pp.1–8，2026.
[11] A. Dalskov, D. Escudero, and M. Keller, “Fantastic four: Honest-Majority Four-Party secure computation with malicious security,” 30th USENIX Security Symposium (USENIX Security 21), pp.2183–2200, 2021.

© 2026 Information Processing Society of Japan